

C++内存管理与智能指针

C++ 内存管理与智能指针自学手册

一、基础概念：内存管理的核心逻辑

1.1 内存区域划分

C++ 程序运行时，内存分为以下区域（按生命周期从长到短排序）：

区域	存储内容	管理方式
静态内存	全局变量、static 局部变量、类静态成员	编译器自动分配 / 释放，程序结束时回收
栈 (Stack)	函数参数、局部变量（非 static）	编译器自动分配 / 释放，遵循“后进先出”（LIFO）原则
堆 (Heap)	动态分配的对象（new/delete 管理）	程序员手动分配 / 释放，需避免内存泄漏和悬挂指针

示例：

```
int global = 10;    // 静态内存
void func() {
    int localVar = 20; // 栈内存
    int* heapVar = new int(30); // 堆内存
}                  // func 结束时，localVar 被释放，heapVar 需手动 delete
```

核心区别：

- 栈**：速度快、容量小（通常几 MB），适合临时数据。
- 堆**：速度慢、容量大（受限于系统内存），适合动态数据。

1.2 动态内存分配的必要性

当以下情况发生时，必须使用堆内存：

1. 对象大小在编译时无法确定（如用户输入数组长度）。
2. 对象生命周期需跨函数 / 作用域（如返回动态分配的指针）。
3. 对象需被多个指针共享（如 `shared_ptr` 管理的资源）。

反模式：

```
int* badFunction() {  
    int localVar = 10;  
    return &localVar; // 错误！返回栈变量地址，调用者使用时会导致未定义行为  
}
```

二、核心知识点：动态内存与智能指针

2.1 手动内存管理：new/delete

基础用法：

```
// 分配单个对象  
int* p1 = new int(42);    // 初始化为 42  
delete p1;                // 释放内存  
// 分配数组  
double* p2 = new double[10]; // 未初始化  
delete[] p2;              // 释放数组（必须用 delete[]）
```

常见错误：

1. 内存泄漏：忘记调用 `delete`。
2. 悬挂指针：释放内存后未置 `nullptr`。
3. 重复释放：对同一指针多次调用 `delete`。

最佳实践：

- 分配与释放严格配对（`new/delete`、`new[]/delete[]`）。
- 释放后立即置 `nullptr`：

```
int* ptr = new int;
delete ptr;
ptr = nullptr; // 避免悬挂指针
```

2.2 智能指针：自动化内存管理

C++11 引入智能指针，基于 RAII（资源获取即初始化）原则，自动释放内存：

类型	核心特性	适用场景
<code>std::unique_ptr</code>	独占所有权，不可拷贝，可移动（ <code>std::move</code> ）	独占资源（如文件句柄、数据库连接）
<code>std::shared_ptr</code>	共享所有权，引用计数管理，可拷贝	多所有者场景（如数据结构节点、共享缓存）
<code>std::weak_ptr</code>	弱引用，不增加引用计数，用于观察 <code>shared_ptr</code> 的状态	解决循环引用（如双向链表）

示例：unique_ptr 独占资源

```
#include <memory>
std::unique_ptr<int> p1(new int(42));
// std::unique_ptr<int> p2 = p1; // 错误！不可拷贝
std::unique_ptr<int> p3 = std::move(p1); // 所有权转移，p1 变为 nullptr
```

示例：shared_ptr 共享资源

```
std::shared_ptr<double> data = std::make_shared<double>(3.14); // 推荐用 make_shared
std::shared_ptr<double> copy = data; // 引用计数增加到 2
```

示例：weak_ptr 解决循环引用

```
struct Node {
    int value;
    std::shared_ptr<Node> next;
    std::weak_ptr<Node> prev; // 弱引用避免循环
};
```

三、进阶用法：动态内存的高级技巧

3.1 自定义删除器

智能指针默认使用 `delete` 释放内存，但可通过自定义删除器处理特殊资源（如文件句柄）：

```
#include <fstream>
void closeFile(std::ofstream* file) {
    file->close();
    delete file;
}
int main() {
    std::unique_ptr<std::ofstream, decltype(&closeFile)> file(
        new std::ofstream("data.txt"),
        closeFile // 自定义删除器
    );
    return 0;
}
```

优势：

- 统一管理资源释放逻辑，避免重复代码。
- 支持非堆资源（如网络连接、锁）的自动释放。

3.2 内存对齐与定位 new

内存对齐：

现代 CPU 要求数据存储在特定地址（如 4 字节、8 字节对齐），否则可能导致性能下降或硬件错误。

```
struct alignas(16) Vector {
    double x, y, z; // 总大小 24 字节，自然对齐到 16 字节
};
```

定位 new：

在指定内存位置创建对象，常用于内存池或嵌入式开发：

```
char buffer[sizeof(Vector)];  
Vector* vec = new (buffer) Vector(1.0, 2.0, 3.0); // 在 buffer 中构造对象  
vec->~Vector(); // 手动调用析构函数
```

四、常见问题与解决方案

4.1 内存泄漏

原因：

- 动态分配的内存未释放。
- 异常导致 `delete` 未执行。

解决方案：

- 用智能指针替代裸指针。
- 使用 `try-finally` 或 RAII 包装资源。

示例： `shared_ptr` 避免异常泄漏

```
void process() {  
    auto resource = std::make_shared<Resource>(); // 异常时自动释放  
    // ... 可能抛出异常的代码 ...  
}
```

4.2 循环引用

场景：

两个 `shared_ptr` 互相引用，导致引用计数无法归零。

```
struct A {  
    std::shared_ptr<B> b;  
};  
struct B {  
    std::shared_ptr<A> a;  
};  
// 创建后，a 和 b 的引用计数均为 1，无法释放  
auto a = std::make_shared<A>();
```

```
auto b = std::make_shared<B>();
a->b = b;
b->a = a;
```

解决方案：

将其中一个指针改为 `weak_ptr`：

```
struct B {
    std::weak_ptr<A> a; // 弱引用不增加计数
};
```

4.3 性能问题

`shared_ptr` 的开销：

- 每个 `shared_ptr` 包含两个指针（对象指针、控制块指针）。
- 引用计数操作（原子操作）可能带来额外开销。

优化建议：

- 优先使用 `unique_ptr` 或 `std::move` 转移所有权。
- 使用 `make_shared` 替代 `new`（减少一次内存分配）。

五、实践案例：从基础到进阶

5.1 动态数组管理

需求： 创建一个动态数组，存储学生成绩并计算平均分。

实现：

```
#include <memory>
#include <vector>
void calculateAverage(int size) {
    std::unique_ptr<int[]> scores(new int[size]);
    for (int i = 0; i < size; i++) {
        scores[i] = i + 1;
    }
}
```

```
double avg = std::accumulate(scores.get(), scores.get() + size, 0.0) / size;
// scores 离开作用域时自动释放
}
```

5.2 资源管理类设计

需求：封装文件操作，确保文件自动关闭。

实现：

```
class File {
public:
    File(const std::string& path) : fp(fopen(path.c_str(), "r")) {
        if (!fp) throw std::runtime_error("File open failed");
    }
    ~File() { fclose(fp); } // RAII 自动关闭文件
private:
    FILE* fp;
};
```

六、练习题与答案

6.1 选择题

1. 以下哪种内存区域由程序员手动管理？

- A. 静态内存
- B. 栈
- C. 堆

答案：C

2. `shared_ptr` 的引用计数何时归零？

- A. 最后一个 `shared_ptr` 被销毁时
- B. 程序结束时
- C. 调用 `reset()` 时

答案：A

3. 以下哪种操作会导致内存泄漏？

- A. `delete[]` 释放 `new` 分配的内存
- B. `delete` 释放 `new[]` 分配的数组
- C. 用 `unique_ptr` 管理动态对象

答案：B

6.2 编程题

题目：设计一个 `Matrix` 类，使用动态内存存储二维数组，要求：

- 1. 支持移动语义，避免深拷贝
- 2. 使用 `unique_ptr` 管理内存
- 3. 提供 `operator()` 访问元素

答案框架：

```
#include <memory>
class Matrix {
public:
    Matrix(int rows, int cols)
        : rows(rows), cols(cols), data(std::make_unique<double[]>(rows * cols)) {}
    // 移动构造函数
    Matrix(Matrix&& other) noexcept
        : rows(other.rows), cols(other.cols), data(std::move(other.data)) {}
    // 禁止拷贝
    Matrix(const Matrix&) = delete;
    Matrix& operator=(const Matrix&) = delete;
    double& operator()(int i, int j) { return data[i * cols + j]; }
private:
    int rows, cols;
    std::unique_ptr<double[]> data;
};
```

七、总结与最佳实践

1. 手动管理内存

- 用 `new/delete` 时，确保配对使用，释放后置 `nullptr`。
- 避免返回栈变量地址。

2. 智能指针选择

- `unique_ptr`：独占资源，优先使用。
- `shared_ptr`：共享资源，配合 `make_shared` 优化性能。
- `weak_ptr`：解决循环引用，需结合 `lock()` 检查有效性。

3. 性能优化

- 用 `std::move` 转移所有权，避免不必要的拷贝。
- 避免过度使用 `shared_ptr`，权衡引用计数开销。

4. 工具与检测

- 使用 Valgrind、AddressSanitizer 检测内存泄漏和悬挂指针。
- 静态分析工具（如 Clang-Tidy）检查潜在问题。

记住：现代 C++ 中，**能用智能指针就不用裸指针**，这是写出安全、高效代码的关键！

附：推荐学习资源

- [C++ 内存管理深度解析](#)
- [智能指针实战教程](#)
- [内存泄漏检测工具对比](#)

通过反复练习和实践，你将逐步掌握内存管理的精髓，成为 C++ 内存大师！

（注：文档部分内容可能由 AI 生成）